

BABYWOLF BLOG

Documento Base del Proyecto — Entregable E1.1

Luis Abraham Camacho Durán Repo: github.com/luuboon/babywolf-blog

1. Funcionalidad de la Aplicación (SA.1)

Problema. Quien sigue el gaming retro no consume contenido de una sola forma: en el teléfono lee un artículo completo, en el reloj quiere ver un dato de un vistazo, y frente a la tele busca algo pasivo navegable con el control. Armar una app por dispositivo sale caro (tres bases de código, inconsistencias, cambios lentos). BabyWolf Blog usa un solo motor para los tres y solo cambia la densidad de información y la navegación según el dispositivo: teléfono (360×740), Wear OS (384×384) y Android TV (1920×1080).

Objetivo general. Desarrollar BabyWolf Blog con Flutter como único motor, para entregar contenido de gaming retro y telemetría de forma cómoda y consistente en teléfono, wearable y TV, aprovechando el backend en Go que ya tengo funcionando.

Objetivos específicos:

1. Lector móvil de artículos con estado vía Provider, que jala el feed desde la API.
2. “Retro Widget” en Wear OS alimentado por BLE: escanear, conectar y leer GATT para mostrar un dato clave.
3. Dashboard para Android TV navegable con D-Pad y contenidos en cuadrícula.
4. Misma identidad visual y reglas de seguridad en las tres plataformas (mismo diseño + JWT).

Alcance — incluye: login y sesión con JWT (Supabase); feed en las tres plataformas; escaneo/conexión/lectura GATT con UI adaptable. Excluye: tienda/pagos, foros o chat, y edición de contenido desde reloj o TV.

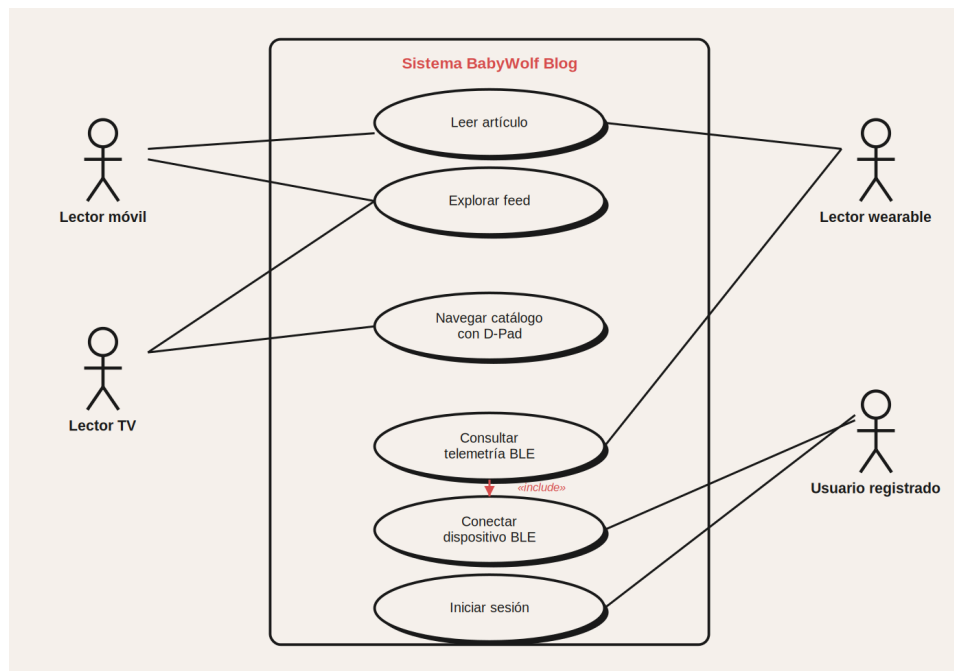


Figura 1. Casos de uso (UML).

Actor	Acción	Cómo lo hace	Resultado
Lector móvil	Leer artículo	Abre la app, recorre el feed, toca una tarjeta y lee el detalle.	Ve el artículo con imagen y texto.
Lector wearable	Ver telemetría	Abre el Retro Widget, escanea, elige dispositivo y lee la característica GATT.	El dato (ej. temperatura/ciudad) aparece de un vistazo.
Lector TV	Recorrer catálogo	Prende la app y se mueve por la cuadrícula con el D-Pad.	Navega a distancia; al elegir abre la lectura.
Usuario registrado	Iniciar sesión	Escribe sus datos; la app valida contra la API y guarda el JWT.	Entra a sus áreas privadas y perfil.

Datos que maneja: perfil de usuario (id, email, username, role, avatar_url — entidad User en Go/PostgreSQL), publicaciones (title, slug, content, cover_image_url, published — entidad Post), preferencias del lector, y telemetría BLE (característica GATT simulada).

Integraciones: API REST en Go 1.25.7 (Gin + GORM); Supabase (PostgreSQL 15, Auth, Storage); hardware BLE vía flutter_blue_plus; correos con Resend.

Viabilidad. Flutter compila a nativo para Android (teléfono y Wear OS) y corre en Android TV sobre la misma base, así que un solo proyecto comparte lógica y diseño; sus plugins nativos resuelven el BLE/GATT. Además el backend en Go ya existe y está probado, por lo que el esfuerzo se concentra en las pantallas, no en rehacer el servidor (ver Figura 2).

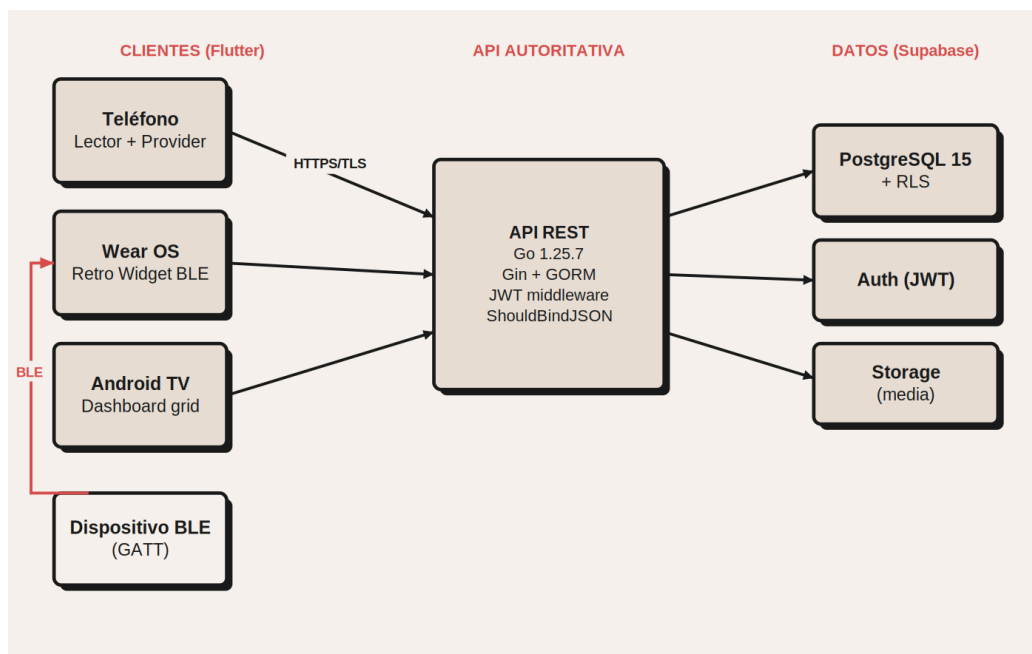


Figura 2. Arquitectura: clientes Flutter → API Go → Supabase.

2. Usabilidad y Accesibilidad (SA.2)

Público objetivo: fans del retro gaming de 16 a 40 años (sobre todo 20–32), con nivel técnico medio-alto, cómodos con apps y dispositivos conectados. Contexto: lectura larga en móvil en ratos libres, consulta rápida en el reloj, y consumo tranquilo en la TV en casa.

Flujo de usuario:

Entra → explora el feed → elige una publicación → lee (puede regresar con atrás). El flujo BLE en el reloj es: abrir Retro Widget → escanear → elegir → conectar → leer → mostrar el dato, con aviso de estado en cada paso (cargando/error/listo).

Accesibilidad. El diseño real usa alto contraste: texto casi negro (#1B1B1B) sobre fondo crema (#F6F2EC), que supera el contraste AA de WCAG (4.5:1) y se lee bien en artículos largos. Los componentes se marcan con bordes gruesos (3–4 px) y sombras duras, así se distinguen sin depender solo del color (ayuda a baja visión y daltonismo). Tamaños mínimos por encima de 12 px: 14 px en móvil y 16 px efectivos en el reloj. Botones grandes y foco visible para D-Pad; texto alternativo en portadas.

Dispositivo	Pantalla	Densidad y layout
Teléfono	360×740	Alta: lista vertical de tarjetas, pestañas, interacción táctil con Provider.
Reloj (Wear OS)	384×384	Mínima: un solo dato, máx. 3 elementos, sin scroll; lectura de un vistazo.
Android TV	1920×1080	Media horizontal: cuadrícula, foco con D-Pad, letra y márgenes grandes.

Pruebas planeadas: probaré la tarea crítica de conexión BLE (escanear → conectar → leer) con al menos 3 personas del público objetivo, midiendo si lo logran, cuánto tardan en el primer dato y cuántos errores cometen; al final una pregunta corta de facilidad y registro de puntos de fricción para ajustar antes del E1.3.

3. Diseño de Interfaces (SA.3)

3.1 Paleta de colores

La paleta sale directo de las variables CSS de styles.scss (estilo neo-brutalista claro). Es una paleta sobria: casi todo el peso visual lo cargan el negro y el crema, con un rojo reservado solo para errores. Las muestras de abajo son los colores tal cual se usan, con su token CSS real.



Figura 3. Paleta de colores y sus tokens CSS (styles.scss).

Para qué	HEX / Token	Por qué
Fondo	#F6F2EC --color-surface	Crema cálido, cansa menos en lectura larga; base para resaltar el texto.
Tinta (texto y bordes)	#1B1B1B --color-primary	Casi negro; es el color de acción (botones, bordes 4 px, sombras). Contraste >12:1 sobre el crema (supera AA y AAA).
Tarjetas / fondo alterno	#E6E0D6 --background-alt	Separa bloques de contenido sin meter más color.
Texto secundario	#333333 --color-text-secondary	Metadatos y subtítulos, manteniendo contraste sobre el crema.
Error / peligro	#DC2626	Único color fuerte de la paleta; solo para alertas y errores (desconexión BLE, validación fallida).

Decisión de diseño: en lugar de un color de acento llamativo, la acción principal se marca con el negro y la sombra dura. Eso mantiene la identidad retro/sobria y reserva el rojo (#DC2626) exclusivamente para errores, de modo que cuando aparece rojo el usuario sabe de inmediato que algo va mal.

3.2 Tipografía

Courier New (monospace) para títulos y cuerpo —es lo que le da el aire retro/técnico de la marca— con Roboto en los componentes de Material. Tamaños mínimos arriba de 12 px (14 px en móvil, 16 px efectivos en el reloj); los títulos van en negritas 700–900, mayúsculas y con espaciado de letra para marcar jerarquía.

3.3 Componentes reutilizables y sus estados

Estos componentes se diseñan una sola vez y se reutilizan en toda la app. Las muestras de abajo reflejan su estilo real (borde de 4 px, sombra dura de 6px y tipografía monospace):



Figura 4. Muestras de componentes con su estilo real (botones, inputs, tarjeta, alerta y carga).

Notas de cada componente:

- Botón primario: fondo negro (#1B1B1B), texto blanco, borde 4 px y sombra dura. Es la acción principal (ej. PUBLICAR).
- Botón secundario: fondo crema con borde negro, para acciones de menor peso (ej. CANCELAR).
- Hover/foco: el botón o tarjeta se recorre un poco y la sombra crece (de 6px a 8px); al presionar se hunde (sombra a 4px). El foco siempre queda visible, importante para navegar con D-Pad en TV.
- Input: borde de 3 px en reposo; al recibir el foco el borde sube a 4 px (“despierta”).
- Tarjeta de contenido (.post-card): portada arriba, título, extracto y metadatos; misma sombra dura.
- Alerta de error: fondo rojo claro con borde y barra lateral #DC2626; aparece si se cae el BLE o falla una validación.
- Cargando: CircularProgressIndicator mientras se escanea el BLE; en el feed se usan skeletons.

Estado	Cómo se ve
Normal	En reposo, con su borde y sombra duros.
Hover / Foco	Se recorre y crece la sombra (tarjetas/botones) o se resalta el borde (inputs); foco visible para D-Pad.
Cargando	CircularProgressIndicator al escanear BLE; skeletons en el feed.
Error	Alerta con rojo #DC2626 si se cae el BLE o falla validación.
Deshabilitado	Baja el contraste y quita la sombra para indicar que no se puede tocar.

3.4 Layout, iconografía y guía de estilo

Layout: teléfono = lista vertical con Provider; reloj = un dato centrado, máx. 3 elementos; TV = cuadrícula con D-Pad y márgenes seguros. Iconografía: set de Material Design como base

+ algunos iconos pixel-art propios en encabezados/pantallas vacías. Guía de estilo: el repo incluye DOCUMENTACION_Y_GUIA_DE_ESTILOS.md, que junta paleta, tipografía, componentes y decisiones para mantener consistencia; es la documentación que usé para registrar la guía de estilos y de qué va el proyecto.

3.5 Mapa de navegación

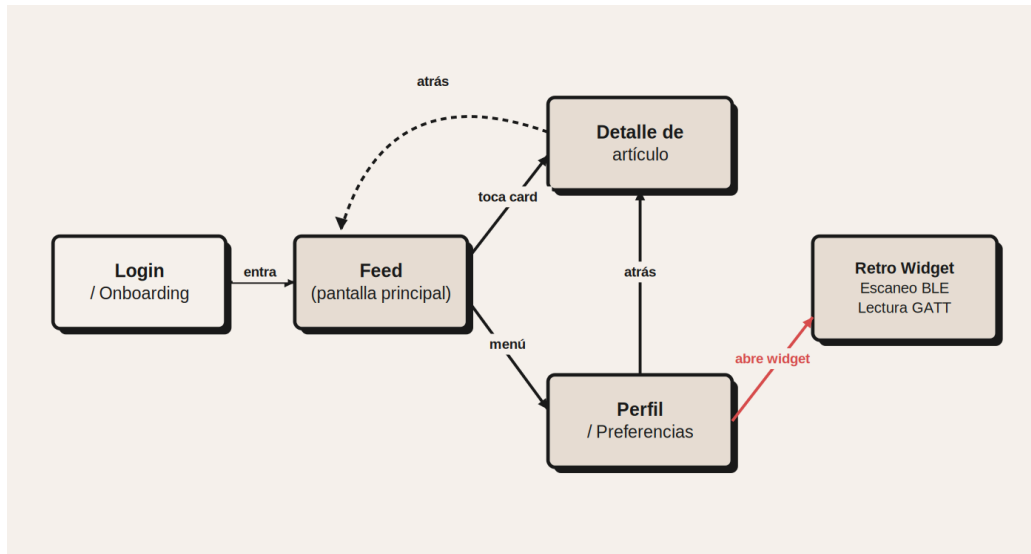


Figura 5. Mapa de navegación entre pantallas.

4. Seguridad Transversal (SA.4)

Leyes. Aplica la LFPDPPP (México) y su reglamento; supletoriamente el Código de Comercio si hay operaciones mercantiles. El diseño respeta licitud, consentimiento, información, calidad, finalidad, proporcionalidad y responsabilidad.

Datos personales identificados: correo (email) y nombre de usuario (username) como PII; identificador, rol y avatar como datos de cuenta; preferencias del lector; y telemetría que llega por BLE (GATT simulada).

Por qué protegerlos. El correo y nombre sirven para suplantación o phishing si se filtran, así que llevan control de acceso estricto. El BLE es un canal físico interceptable, por eso aplico mínima exposición: el reloj recibe solo el dato necesario.

Almacenamiento. Los datos viven en PostgreSQL en Supabase con Row Level Security (confianza cero): cada consulta se filtra por el token antes de devolver nada, un usuario solo toca su propia fila y los borradores (published = false) no los ve nadie. Las imágenes van en Supabase Storage con RLS. En el dispositivo solo se guarda el token de sesión y preferencias no sensibles.

Comunicaciones. Todo entre la app y la API va por HTTPS con TLS 1.2+. La sesión viaja en Authorization: Bearer <JWT>, y el middleware del backend (SupabaseAuthMiddleware) verifica la firma HMAC y la vigencia del token antes de dejar pasar. El BLE usa el emparejamiento y cifrado del propio protocolo.

Credenciales. Los secretos (JWT_SECRET, llaves de Supabase) se manejan con variables de entorno vía godotenv y nunca van en el código. El .gitignore está puesto antes del primer commit para excluir los .env del historial de Git.

Validación. Dos capas: en Flutter los formularios revisan y limpian antes de enviar; en el backend (Go) cada endpoint valida el cuerpo con ShouldBindJSON y responde 400 si los datos vienen mal, así nada sin validar llega a la base.

Aviso de privacidad (borrador). BabyWolf Blog recolecta tu correo y nombre de usuario para tu cuenta e inicio de sesión, y tus preferencias para personalizar contenido. En telemetría, lee datos del dispositivo Bluetooth que elijas y conserva solo el dato necesario. No vende ni comparte datos con terceros para publicidad. Puedes ejercer tus derechos ARCO por los medios de la app. Todo conforme a la LFPDPPP.

Certificación de acceso. En el onboarding la app muestra el aviso y pide consentimiento explícito con una casilla (checkbox) que debe marcarse antes de crear la cuenta o dar permiso de Bluetooth; el consentimiento queda ligado a la cuenta como constancia de autorización informada, incluidas las pruebas.

5. Declaración de Uso de Inteligencia Artificial

Conforme a la sección 4.2 de la Política de Uso de IA del curso, declaro de forma transparente cómo utilicé herramientas de IA en este entregable.

5.1 Herramienta y para qué la usé

Herramienta: Claude. La usé en dos roles, ambos permitidos por la política:

Uso	En qué consistió
Estructurar y redactar	Organizar el documento según la rúbrica (SA.1–SA.4) y redactar la prosa a partir de la información de mi proyecto.
Extraer datos de mi repo	Leer mi repositorio público para tomar los datos técnicos reales: paleta de colores (styles.scss), entidades del backend (User, Post), stack (Go/Gin/GORM, Supabase) y patrones de los componentes.

5.2 Qué es mío y puedo defender

El proyecto, el código y las decisiones de diseño y arquitectura son míos. La IA no tomó decisiones por mí: partió de lo que ya construí. Puedo justificar en la demo, entre otras cosas:

- Por qué la paleta neo-brutalista clara (#F6F2EC / #1B1B1B), con el rojo #DC2626 reservado solo para errores, y por qué Courier New; y qué problema tendría un usuario con daltonismo (resuelto con bordes y sombras, no solo color).
- Por qué un solo motor (Flutter) en lugar de apps separadas, y por qué un backend dedicado en Go además de Supabase.
- Qué pasa si el companion pierde la conexión BLE (estado de error visible y reintento), y dónde está en el código la validación de entrada.

5.3 Qué no se usó

No usé IA para generar evidencia falsa, capturas de procesos que no realicé ni conclusiones de pruebas inexistentes (las pruebas de usabilidad están declaradas como planeadas, no como hechas). Revisé y procesé todo el contenido antes de entregarlo; el texto pasó por mi criterio y refleja mi proyecto real.